CS-499-12008-M01

Brendan Richardson

08 Feb 2026

CS-499 Milestone Three

The artifact I enhanced for this milestone is my CS-300 Project Two file. It is a C++ console based course planner program for “ABCU”. I originally created this project in 2023 as part of my Data Structures and Algorithms course. The program reads course data from a CSV file, stores course information in memory, and allows the user to print a sorted course list or look up a specific course and its prerequisites.

I selected this artifact for my ePortfolio because it provides a strong foundation for demonstrating core algorithms and data structures skills in a realistic use case. The course planner is essentially a data driven lookup and validation system. It loads records, supports search, validates relationships, and outputs results for the user. Those are exactly the types of tasks that come up when working with structured data in professional settings, especially in analytics workflows where clean input, reliable relationships, and efficient querying matter.

To strengthen the artifact, I implemented enhancements that go beyond basic lookup and sorting. First, I improved prerequisite handling by storing prerequisites in an unordered_set rather than a simple list structure, which enforces uniqueness and supports faster membership operations. After that I implemented a graph based validation feature that builds an adjacency list of course prerequisite relationships and uses a depth-first search cycle detection algorithm to identify circular prerequisite chains. This adds a meaningful correctness layer that the original program did not have. Finally, I added a prefix search feature supported by a sorted index and binary search (lower_bound), allowing the user to efficiently find and display courses that share

a common prefix, like all “CSCI” courses. Together, these changes make the program more scalable, more reliable, and more aligned with real world data validation and retrieval needs.

These enhancements align well with the course outcome focused on designing and evaluating computing solutions using algorithmic principles while managing trade offs. The DFS cycle detection feature demonstrates algorithmic reasoning and the ability to model data as a graph to solve a relationship based problem. The prefix search feature demonstrates how sorting and binary search can be applied to support faster query operations, rather than relying on manual scanning. The use of unordered_set and unordered_map also demonstrates practical data structure selection to improve performance and data integrity. At this point, I don’t need to change my outcome-coverage plan, but this milestone strengthens my coverage by demonstrating more advanced algorithmic work beyond basic searching and printing.

Enhancing this artifact helped me see how data structures and algorithms are not just academic concepts, but tools that directly affect performance and correctness in software that processes structured data. A major takeaway for me was that validating the quality of data relationships is just as important as storing and retrieving records. Implementing cycle detection forced me to think about prerequisites as a graph rather than isolated values, and it made the system more trustworthy. The prefix search enhancement also reinforced the value of building indexes and using efficient search algorithms rather than relying on linear scans that were just good enough.

The most challenging part of the enhancement process was keeping the changes substantial while still protecting the program’s original behavior and requirements. Adding advanced logic like graph validation and binary search based prefix searching required careful planning so the program still loads files, prints courses, and retrieves course information

correctly. I also had to pay attention to consistency issues, such as normalizing course IDs to uppercase to prevent mismatches between user input and stored data. Overall, these enhancements improved the program's efficiency and reliability while giving me stronger evidence of algorithmic problem solving for my ePortfolio.

```
C:\Users\brend\source\repos\

ABCU COURSE PLANNER
[1] Load Data Structure.
[2] Print Course List.
[3] Print Course.
[4] Search Courses by Prefix.
[5] Validate Prerequisites.
[9] Exit

What would you like to do? 1
Enter the file name: C:\Users\brend\source\repos\ProjectTwo\CS 300 ABCU_Advising_Program_Input.csv
Courses loaded successfully (8 records).
Prerequisite validation passed (no cycles detected)

[1] Load Data Structure.
[2] Print Course List.
[3] Print Course.
[4] Search Courses by Prefix.
[5] Validate Prerequisites.
[9] Exit

What would you like to do? 2

Here is a sample schedule:

CSCI100, Introduction to Computer Science
CSCI101, Introduction to Programming in C++
CSCI200, Data Structures
CSCI300, Introduction to Algorithms
CSCI301, Advanced Programming in C++
CSCI350, Operating Systems
CSCI400, Large Software Development
MATH201, Discrete Mathematics

[1] Load Data Structure.
[2] Print Course List.
[3] Print Course.
[4] Search Courses by Prefix.
[5] Validate Prerequisites.
[9] Exit

What would you like to do? 4
Enter prefix (example: CSCI): CSCI
Matches:
CSCI100, Introduction to Computer Science
CSCI101, Introduction to Programming in C++
CSCI200, Data Structures
CSCI300, Introduction to Algorithms
CSCI301, Advanced Programming in C++
CSCI350, Operating Systems
CSCI400, Large Software Development

[1] Load Data Structure.
[2] Print Course List.
[3] Print Course.
[4] Search Courses by Prefix.
[5] Validate Prerequisites.
[9] Exit

What would you like to do? 5
No circular prerequisites detected.

[1] Load Data Structure.
[2] Print Course List.
[3] Print Course.
[4] Search Courses by Prefix.
[5] Validate Prerequisites.
[9] Exit

What would you like to do?
```